MNIST Image Classification with a Restricted Boltzmann Machine and a Logistic Regression Classifier

1 DATASET ORIGIN AND EXPLANATION

The dataset used for this project is the MNIST dataset, a dataset with grayscale images of handwritten numbers 0-9. It is loaded directly from the Tensorflow package.

1.1 TECHNICAL DESCRIPTION

Restricted Boltzmann machines are useful to derive the components of a sample. They are a machine learning model that can be trained with unlabeled data. This can be a numerical sample describing any observation such as a set of measurements or an image. With these components, RBMs can detect anomalies in a set of images, or use the components to train a classifier. The latter one was done and is explained in this report.

The TensorFlow library was used to create a Restricted Boltzmann Machine to classify a color image into its components. This model was changed from the previous delivery to adapt to the correctly defined problem.

1.2 ALGORITHM DESIGN

1.2.1 Data Preparation

The dataset was loaded directly from the tensorflow library into the X and y already split into train and test. The image pixel values were normalized to a value between 0 and 1 and their shaped was flattened so that each image was a 1-D array.

All the data from all digits is passed to the model because the objective is for it to discern the different numbers. If we had wanted anomaly detection, only one of the numbers would have been passed to the model.

1.2.2 Model Architecture

The model built is a RBM feature extractor with a Logistic Regression that classifies through the extracted features. The RBM has a set of visible nodes (the input ones) and a set of hidden ones, which represent the features it extracts from the inputs. The model has a matrix containing the weights of the relationships between inputs and features, as well as biases for each input and hidden node.

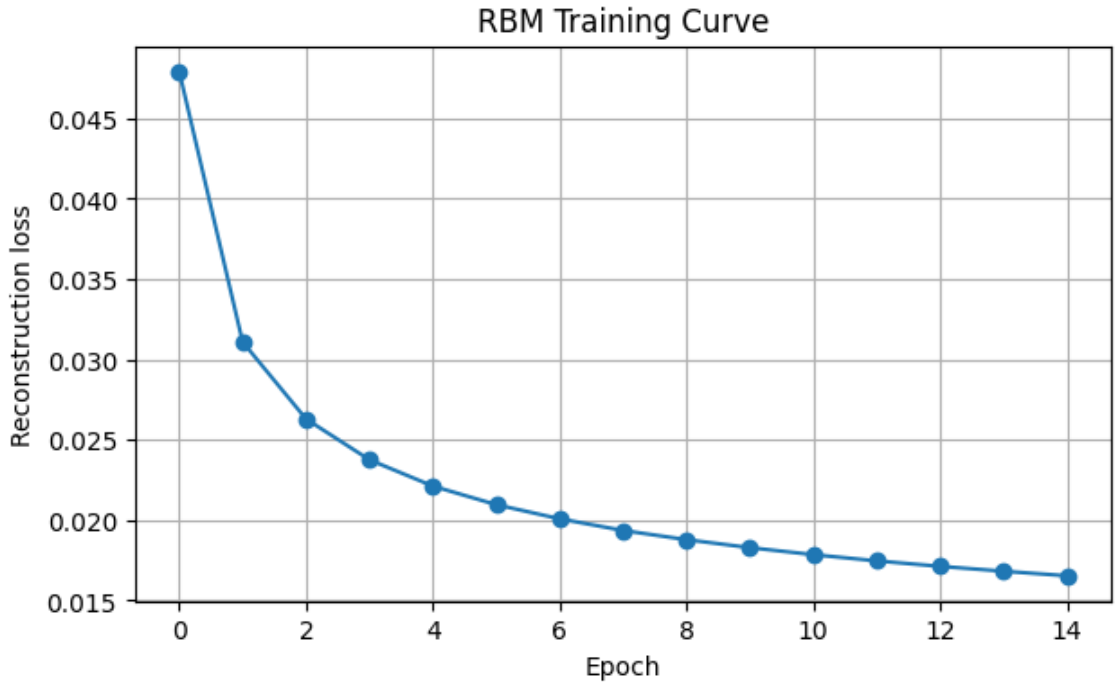
The model has a forward pass where the visible units are transformed into hidden feature probabilities using the weights. This captures patterns in the data. Then, there is a backward pass where these hidden features are used to reconstruct the visible units, producing an approximation of the original input, in this case, the reconstructed image. After the reconstruction, the model adjusts the weights and biases so that the difference between the original image and the reconstructed one is minimized.

After this is done and the first model is trained, the features of the model are calculated. The Logistic Regression model now takes a new sample and passes it through the model, this time only doing the forward pass. The values that would be used to reconstruct the image in training, are instead passed into the logistic regression model for training this time, and the same will be done when classifying a new image to test it.

1.2.3 Configurations

There initial weights are set randomly, and later updated through the training process. Starting multiple times gives very similar results, showing that there is no advantage in setting specific ones. A gradient is calculated to update the weights during the training process, through Contrastive Divergence which is a method that approximates the gradient without making a full backpropagation.

The RBM was trained for 15 epochs, when the model's loss started to plateau, as can be seen in the following figure.



1.2.4 Alternative Model

Two Logistic Regression models were built, one being fed the RBM's features and the other the raw image data.

1.3 Results of the Algorithms and Analysis of the Results

Using the features from the RBM, the model achieved an accuracy of 0.9441, while the raw data achieved an accuracy of 0.9264.

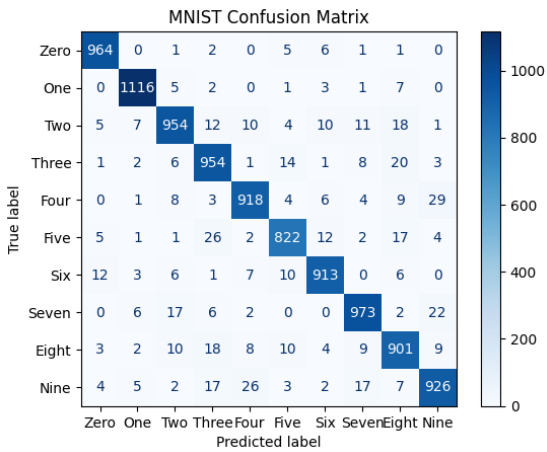


Figure 1. With RBM

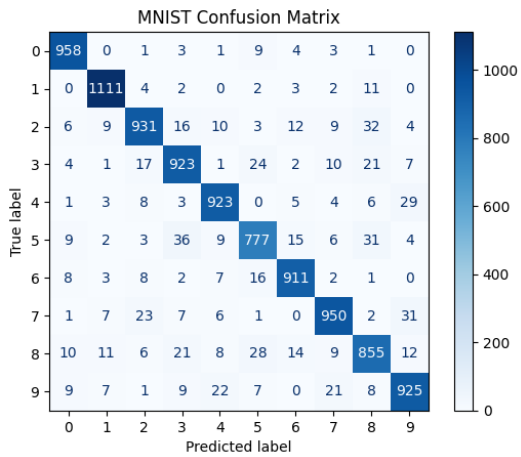


Figure 2. Without RBM

These results indicate towards a slightly better model using the RBM as a feature extraction model, with improvements in the result of almost all numbers in the MNIST dataset.

This result could be expected, since it may be easier for a simple model like Logistic Regression to learn from processed features rather than only pixels. Despite this, it comes at a cost of training a second model (the RBM) and using it every time we want to make a prediction.

1.4 Conclusions

A simple classification model can benefit from an unsupervised model that processes the raw data before the classification model. RBMs are good at anomaly detection but can also serve this purpose. Thanks to their unsupervised nature, they encode the data into features that can be more closely related to the label we want to predict than the raw data. This project demonstrated this

1.5 Code

```
# Load mnist from Keras
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
class_names = [
    "Zero", "One", "Two", "Three", "Four",
    "Five", "Six", "Seven", "Eight", "Nine"
]

# normalize and change shape to work with model
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32")  / 255.0
x_train = x_train.reshape(len(x_train), -1)
x_test  = x_test.reshape(len(x_test),  -1)
y_train = y_train.flatten()
y_test  = y_test.flatten()

# create the dataset from the data in batches of 128
train_ds = (tf.data.Dataset.from_tensor_slices(x_train.astype(np.float32))
    .shuffle(5000, seed=42).batch(128).prefetch(tf.data.AUTOTUNE))

# declare the RBM class, with a constructor and the necessary methods to train the model as
explained in the model architecture and configuration sections.
class RBM(tf.Module):
    def __init__(self, n_visible, n_hidden, learning_rate=0.01, name=None):
        super().__init__(name=name)
        self.n_visible = n_visible
        self.n_hidden = n_hidden
```

```
        self.learning_rate = learning_rate

        initializer = tf.random.normal([n_visible, n_hidden], mean=0.0, stddev=0.01)
        self.W = tf.Variable(initializer, name="W")
        self.b_v = tf.Variable(tf.zeros([n_visible]), name="b_v")
        self.b_h = tf.Variable(tf.zeros([n_hidden]), name="b_h")

    def hidden_prob(self, v):
        return tf.nn.sigmoid(tf.matmul(v, self.W) + self.b_h)

    def visible_prob(self, h):
        return tf.nn.sigmoid(tf.matmul(h, tf.transpose(self.W)) + self.b_v)

    def sample_bernoulli(self, probs):
        random_uniform = tf.random.uniform(tf.shape(probs))
        return tf.cast(random_uniform < probs, tf.float32)

    def contrastive_divergence(self, v0):
        h0_prob = self.hidden_prob(v0)
        h0_sample = self.sample_bernoulli(h0_prob)

        v1_prob = self.visible_prob(h0_sample)
        v1_sample = self.sample_bernoulli(v1_prob)

        h1_prob = self.hidden_prob(v1_sample)

        batch_size = tf.cast(tf.shape(v0)[0], tf.float32)

        positive_grad = tf.matmul(tf.transpose(v0), h0_prob)
        negative_grad = tf.matmul(tf.transpose(v1_sample), h1_prob)

        dW = (positive_grad - negative_grad) / batch_size
        db_v = tf.reduce_mean(v0 - v1_sample, axis=0)
        db_h = tf.reduce_mean(h0_prob - h1_prob, axis=0)

        self.W.assign_add(self.learning_rate * dW)
        self.b_v.assign_add(self.learning_rate * db_v)
        self.b_h.assign_add(self.learning_rate * db_h)

        recon_error = tf.reduce_mean(tf.square(v0 - v1_prob))
        return recon_error

    def reconstruct(self, v):
        h_prob = self.hidden_prob(v)
        v_prob = self.visible_prob(h_prob)
        return v_prob


# create the model class with the corerct input shape
n_visible = 784
n_hidden = 128
learning_rate = 0.05
epochs = 15

rbm = RBM(n_visible=n_visible, n_hidden=n_hidden, learning_rate=learning_rate)

# train the model for 15 epochs
history = []

for epoch in range(1, epochs + 1):
    epoch_errors = []

    for batch in train_ds:
        err = rbm.contrastive_divergence(batch)
        epoch_errors.append(float(err.numpy()))

    mean_err = np.mean(epoch_errors)
    history.append(mean_err)
    print(f"Epoch {epoch:02d}/{epochs} - reconstruction loss: {mean_err:.6f}")

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
# Get hidden representations
def get_hidden_features(rbm, x):
    h = rbm.hidden_prob(tf.constant(x.astype(np.float32)))
    return h.numpy()

X_train_feat = get_hidden_features(rbm, x_train)
X_test_feat = get_hidden_features(rbm, x_test)

clf = LogisticRegression(max_iter=1000)

clf.fit(X_train_feat, y_train.flatten())

y_pred = clf.predict(X_test_feat)

print("Accuracy:", accuracy_score(y_test, y_pred))

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# compute matrix
cm = confusion_matrix(y_test, y_pred)

# digit names
labels = [class_names[i] for i in range(10)]

# plot
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="Blues", values_format="d")

plt.title("MNIST Confusion Matrix")
plt.show()


# this same code above was used with the raw data model
```